



Lecture 03 : Data Structures

So far, you have been working with fundamental data types like str, int, and float. Many real-world problems are easier to solve when simple data types are combined into more complex data structures

Recap: note on print() function:

You've used print() many times already, especially in the homework. It can print one or multiple objects. By default, these are separated by a whitespace and end with a line break:

```
In [ ]: print("hello", "there")
```

As you can see, the whitespace gets added between the two strings. A second print() statement would also print to a new line, since this one ended on the new line character. For print(), this behaviour is controlled via the parameters "sep" and "end". Via these parameters we can change how the print function behaves, which may be useful in some specific use cases:

```
In [ ]: print("hello", "there", sep="", end="!")
```

Note how now that we did not put "\n" but "!" as the end parameter, there is no new line before our next input.

Data Structure

A data structure models a collection of data, such as a list of numbers, a row in a spreadsheet, or a record in a database.

Python has three built-in data structures that are the focus of this chapter: tuples, lists and dictionaries.

Tuples

Perhaps the simplest compound data structure is a sequence of items.

A sequence is an ordered list of values. Each element in a sequence is assigned an integer, called an **index**, that determines the order in which the values appear.

Just like strings, the index of the first value in a sequence is 0.

Tuples are ordered because their elements appear in an ordered fashion. The first element of (1, 2, 3) is 1, the second element is 2, and the third is 3.

A tuple literal is a tuple that is written out explicitly as a comma-separated list of values surrounded by parentheses:

```
In [ ]: tuple1 = (1, 2, 3)
```

```
In [ ]: type(tuple1)
```

Tuples and strings have a lot in common. The main difference between strings and tuple is unlike strings, which are sequences of characters, tuples may contain **any type of value**, including values of different types.

```
In [ ]: tuple2 = (1, "John", "Red")
```

```
In [ ]: tuple2
```

An empty tuple is a special tuple that doesn't contain any values:

```
In [ ]: tuple3 = ()
```

Tuples Have a Length:

```
In [ ]: len(tuple2)
```

Tuples Support Indexing and Slicing:

```
In [ ]: tuple2[1]
```

Like strings, tuples are **immutable**. This means you can't change the value of an element of a tuple once it has been created.

```
In [ ]: tuple2[1] = "David"
```

Tuples Are Iterable (you can loop over them):

```
In [ ]: for item in tuple2:
    print(item)
```

Checking Existence of Values With in

```
In [ ]: if "Red" in tuple2:
    print('There is red')
else:
    print('There is no red')
```

Lists

The list data structure is another sequence type in Python.

Just like strings and tuples, lists contain items that are indexed by integers, starting with 0.

A list literal looks almost exactly like a tuple literal, except that it is surrounded with square brackets [and] instead of parentheses:

```
In [ ]: mylist1 = [1,2,3]
```

```
In [ ]: type(mylist1)
```

Like tuples, lists values are not required to be of the same type:

```
In [ ]: mylist2 = [1, "John", "Red"]
```

```
In [ ]: mylist2
```

You can create a list from a string of a comma-separated list of items using the string object's **.split()** method:

```
In [ ]: groceries = "eggs, milk, cheese"
grocery_list = groceries.split(", ")
```

```
In [ ]: grocery_list
```

Indexing and slicing operations work on lists the same way they do on tuples:

```
In [ ]: grocery_list[1:]
```

Unlike tuples, however, lists are **mutable**, meaning you can change the value at an index even after the list has been created.

```
In [ ]: grocery_list[1] = 'butter'
```

```
In [ ]: grocery_list
```

List methods

Although you can add and remove elements with slice notation, list methods provide a more natural and readable way to mutate a list.

The **list.insert()** method is used to insert a single new value into a list. It takes two parameters, an index i and a value x, and inserts the value x at index i in the list:

```
In [ ]: mylist2 = []
```

```
In [ ]: mylist2.insert(0, 'A')
```

```
In [ ]: mylist2
```

```
In [ ]: mylist2.insert(2, 'C')
```

```
In [ ]: mylist2
```

```
In [ ]: mylist2.insert(1, 'B')
```

```
In [ ]: mylist2
```

The **list.pop()** method takes one parameter, an index i, and removes the value from the list at that index. The value that is removed is returned by the method:

```
In [ ]: out = mylist2.pop(2)
```

```
In [ ]: out
```

```
In [ ]: mylist2
```

Unlike .insert(), Python raises an IndexError if you pass to .pop() an argument larger than the last index:

```
In [ ]: out = mylist2.pop(3)
```

The **list.append()** method is used to append a new element to the end of a list:

```
In [ ]: mylist2.append('D')
```

```
In [ ]: mylist2
```

.append() is equivalent to inserting an element at an index greater than or equal to the length of the list. What will the the general equivalent to append using .insert()?

```
In [ ]:
```

The **list.extend()** method is used to add several new elements to the end of a list:

```
In [ ]: mylist2.extend(['E','F'])
```

```
In [ ]: mylist2
```

List of numbers

Consider a list of numbers:

```
In [ ]: mylist3 = [4, 17, 42, 5]
```

One very common operation with lists of numbers is to add up all the values to get the total:

```
In [ ]: total = 0
for number in mylist3:
    total = total + number

total
```

Although this for loop is straightforward, there is a much more succinct way of doing this in Python:

The built-in **sum()** function takes a list as an argument and returns the total of all the values in the list:

```
In [ ]: sum(mylist3)
```

Besides sum(), there are two other useful built-in functions for working with lists of numbers: **min()** and **max()**.

Nesting, Copying, and Sorting

Now that you have learned what tuples and lists are, how to create them, and some basic operations with them, let's look at three more concepts:

1. Nesting
2. Copying
3. Sorting

Nesting Lists and Tuples

Lists and tuples can contain lists and tuples as values.

A nested list, or nested tuple, is a list or tuple that is contained as a value in another list or tuple.

For example, the following list has two values, both of which are other lists:

```
In [ ]: my_nested_list = [[1, 2], [3, 4]]
```

```
In [ ]: my_nested_list
```

```
In [ ]: my_nested_list[1]
```

You can use **double index** notation to access an element in the nested list.

```
In [ ]: my_nested_list[1][0]
```

Readers interested in data analysis or scientific computing may recognize lists of lists as a sort of **matrix** of values.

Copying a List

Sometimes you need to copy one list into another list. However, you can't just reassign one list object to another list object, because you'll get this (possibly surprising) result:

```
In [ ]: animals = ["lion", "tiger", "cheetah"]
large_cats = animals
large_cats.append("Leopard")
```

```
In [ ]: large_cats
```

```
In [ ]: animals
```

A variable name is really just a reference to a specific location in computer memory. Instead of copying all the contents of the list object and creating a new list, large_cats = animals assigns the memory location referenced by animals to large_cats. That is, both variables now refer to the same object in memory, and any changes made to one will affect the other.

To get an independent copy of the animals list you can use slicing notation to return a new list with the same values:

```
In [ ]: animals = ["lion", "tiger", "cheetah"]
large_cats = animals[:]
large_cats.append("Leopard")
```

```
In [ ]: large_cats
```

```
In [ ]: animals
```

Sorting Lists

Lists have a **.sort()** method that sorts all of the items in ascending order. [Inplace]

```
In [ ]: numbers = [1, 10, 5, 3]
```

```
In [ ]: numbers.sort()
```

```
In [ ]: numbers
```

Available for list only:

```
In [ ]: sorted(numbers, reverse=True)
```

For example, to sort a list of strings by the length of each string, you can pass the len function to **key**:

```
In [ ]: colors = ["red", "yellow", "green", "blue"]
```

```
In [ ]: colors.sort(key=len)
```

```
In [ ]: colors
```

Dictionaries

One of the most useful data structures in Python is the **dictionary**.

Python dictionaries, like lists and tuples, store a collection of objects.

However, instead of storing objects in a sequence, dictionaries hold information in pairs of data called **key-value pairs**.

That is, each object in a dictionary has two parts: a **key** and a **value**.

The **key** in a key-value pair is a unique name that identifies the value part of the pair.

The following code creates a dictionary literal containing names of countries and their capitals:

```
In [ ]: capitals = {
    "France": "Paris",
    "Spain": "Madrid",
    "Italy": "Rome",
}
```

Notice that each key is separated from its value by a colon (:), each key-value pair is separated by a comma (,), and the entire dictionary is enclosed in curly braces { and }.

```
In [ ]: capitals
```

You can create an empty dictionary using either a literal or dict():

```
In [ ]: dict_example = {}
```

```
In [ ]: dict_example
```

Accessing Dictionary Values To access a value in a dictionary, enclose the corresponding key in square brackets [and] at the end of dictionary or a variable name assigned to a dictionary:

```
In [ ]: capitals["France"]
```

dictionaries are a fundamentally different data structure than sequence types like lists and tuples, how? Values in a sequence type are accessed by index, which is an integer value expressing the order of items in the sequence. On the other hand, items in a dictionary are accessed by a key, which doesn't define any kind of order, but just provides a label that can be used to reference the value.

Adding and Removing Values in a Dictionary You can add and remove items from a dictionary. Like lists, dictionaries are mutable data structures.

Let's add a new capital to the capitals dictionary:

```
In [ ]: capitals["Germany"] = "Berlin"
```

```
In [ ]: capitals
```

To remove an item from a dictionary, use the del keyword with the key for the value you want to delete:

```
In [ ]: del capitals["Spain"]
```

```
In [ ]: capitals
```

Checking the Existence of Dictionary Keys if you try to access a value in a dictionary using a key that doesn't exist, Python raises a **KeyError**:

```
In [ ]: capitals["Spain"]
```

You can check that a key exists in a dictionary using the **in** keyword:

```
In [ ]: if "Spain" in capitals.keys():
    print( capitals["Spain"] )
```

Iterating Over Dictionaries Like lists and tuples, dictionaries are iterable. However, looping over a dictionary is a bit different than looping over a list or tuple.

When you loop over a dictionary with a for loop, you iterate over the **dictionary's keys**:

```
In [ ]: for key in capitals:
    print(key)
```

```
In [ ]: for country in capitals:
    print(f'The capital of {country} is {capitals[country]}')
```

However, there is a slightly more succinct way to do this:

```
In [ ]: capitals.items()
```

```
In [ ]: for country, capital in capitals.items():
    print(f'The capital of {country} is {capital}')
```

How to Pick a Data Structure?

Use a **list** when:

- Data has a natural order to it
- You will need to update or alter the data during the program
- The primary purpose of the data structure is iteration

Use a **tuple** when:

- Data has a natural order to it
- You will not need to update or alter the data during the program
- The primary purpose of the data structure is iteration

Use a **dictionary** when:

- The data is unordered, or the order does not matter
- You will need to update or alter the data during the program
- The primary purpose of the data structure is looking up values

Sets

Finally, short note on sets. Sets are used to store multiple items in a single variable.

A set is a collection which is unordered, unchangeable, and unindexed.

Sets cannot have two items with the same value.

```
In [ ]: myset = {"apple", "banana", "cherry"}
```

```
In [ ]: myset
```

EXERCISES 01 - PART FIVE

List Comprehensions

A way to create a list from an existing iterable is with a list comprehension:

```
In [ ]: numbers = [1, 2, 3, 4, 5]
squares = [num**2 for num in numbers]
```

```
In [ ]: squares
```

A list comprehension is a short-hand for a for loop. In the example above, a tuple literal containing five numbers is created and assigned to the numbers variable. On the second line, a list comprehension loops over each number in numbers, squares each number, and adds it to a new list called squares.

A traditional for loop is:

```
In [ ]: squares = []
for num in numbers:
    squares.append(num**2)
```

```
In [ ]: squares
```

List comprehensions are commonly used to convert values in one list to a different type.

For instance, converting a list of strings containing floating point values to a list of float objects:

```
In [ ]: str_numbers = ["1.5", "2.3", "5.25"]
float_numbers = [float(value) for value in str_numbers]
```

```
In [ ]: float_numbers
```

EXERCISES 02 - PART ONE, TWO and THREE

EXERCISES 01

```
In [ ]:
```